

PRD: MK Internal Timekeeping and Compliance App (POC)

Prepared for: Steve, Director of Growth

Purpose: Hand-off document for the internal build team (Claude Code)

Status: Draft v2 — updated 14 Aug 2026, live in production

Scope: Interim internal tool. Replaces three manual spreadsheets until the third-party HR software pilot concludes. Built as a proof of concept; since grown into the team's actual day-to-day system of record while remaining designed for export, not permanence.

Status update (v2, 14 Aug 2026)

v1 (below, largely unchanged as the historical record of what was originally scoped and built) covered PRD §4–§8 and the acceptance test. Since then the app has been deployed to real production infrastructure and grown well past the original nine screens, based on direct manager/employee feedback rather than a new requirements round. This section is the honest current state; where it conflicts with wording further down in this document, this section wins.

What's live that wasn't in v1:

- **Break tracking with a live timer** (Start/End Break, Personal or Lunch/Dinner), netted out of gap-flagging and blocked from overlapping logged task time. A completed break auto-adds a read-only "General / Break" row to the task log (so it's visible without an employee re-typing it under a real client's name) with an optional, editable note — display-only, it never inflates the logged total.
- **Punch In/Out** — a personal countdown-to-target widget, separate from task logging, that pauses during breaks and flips to counting overtime once the target is reached. Feeds an **automatic Punch-Clock compensation balance** (My Month) independent of the manual compensation-link feature in §5.
- **Overtime pre-approval requests** — an employee requests approval for a future date range from their Team Lead (or any Super Admin if unassigned); Reports → Attendance shows both raw and lead-approved overtime, so payroll can see what's actually payable.
- **A three-tier admin role** (Employee / department-scoped Admin / Super Admin) instead of the flat two-role model in §3 below, plus an independent Developer flag gating the Ticketing System.
- **The Ticketing System** — anyone can raise a bug/enhancement/new-feature ticket, view every ticket, and comment; a Developer can change its status. Built and tested, currently behind a feature flag pending its own rollout.
- **Employee self-service leave requests** (resolves open question 5 below) with a live per-type leave balance (annual entitlement minus approved leave taken this calendar year, resets each January) — admin still enters leave directly too, and still approves/rejects employee-submitted requests.
- **Support questions** (ask an admin, get a reply) as a separate, simpler channel from the Ticketing System.
- **Personal Details and Employment Details self-service profile cards** — DOB, contact info, family, and bank/PAN/Aadhaar/UAN/ESI numbers (masked to last-4 everywhere, including to admins, the instant they're saved), plus profile photo upload.

- **Bulk employee onboarding/updating/offboarding, leave-quota, and Project/Task-list management, all via Excel upload** — not just the roster.
- **Three Reports pages** (Attendance, Strikes, Time by Project/Task with a monthly trend view) with cascading Department → Employee → date-range filters and XLSX export, replacing the single “any view to CSV/XLSX” line in §7.
- **Company Holiday Management** — an admin-maintained calendar (add inline or bulk-upload Name + Date), one shared list for every employee. Resolves open question 9 below; a brief attempt at splitting it by employee country (US/India) was tried and reverted within days once real usage showed the team wanted one common calendar.
- **A rejected task-log submission no longer resets the form** — whatever was typed re-shows, and a time conflict gets an auto-suggested fix, rather than the employee retyping everything from scratch.

What changed from the original build guidance (§9):

- **Hosting is Google Cloud Platform, not Azure App Service** — Cloud Run (service `mk-timekeeping`, region `asia-south1`) with Cloud SQL for PostgreSQL (`mk-timekeeping-pg`) and a Cloud Storage bucket for profile-photo persistence across deploys. An equivalent Azure deployment script (`deploy_azure.sh`) is kept in the repo as a documented, non-one-way-door migration path, but GCP is what’s actually live. The original “Azure App Service, same tenant as email” reasoning no longer applied once the team decided to stand up its own GCP project instead of waiting on the org’s Azure tenant.
- **Auth is self-signup password auth, not Entra ID** — `AUTH_MODE=password`: an admin creates the roster row, the employee claims it via `/signup` against their own email, with rate-limited lockout after repeated failures. Three Super Admins (Deepthi, Steve, Norine) are bootstrapped on first deploy via `BOOTSTRAP_ADMINS` so someone can sign in on a brand-new, empty database. `app/auth.py` still keeps Entra ID as a same-tenant swap-in for later (`AUTH_MODE=entra`) if the org ever standardizes on it — nothing else in the app would need to change.
- **Postgres, not SQLite, in production** — SQLite remains the zero-setup local-dev default; `DATABASE_URL` switches the same code to Postgres with no code changes.

None of this changes the core model from §6/§8 below: statuses, variance, and strikes are still computed, never typed, and the acceptance test (§12.3, reproducing the legacy sheets’ strike counts) still passes 168/168.

1. Problem

HR/admin runs offshore time tracking across three manually maintained spreadsheets. The process works but does not scale, and the files themselves are degrading.

Artifact	What it does today	Verified pain point
Task Summary (one file per person)	Every task logged to the minute: Date, Project/Employer, Task, Details, Start Time, End Time, Time taken, Total Time taken. Employee sums total hours at end of day.	One person's file (Divya, 622 task rows since March 2026) is 20 MB and decompresses to a 700 MB XML grid with 16,331 empty repeated columns per row. Files are slow to open, hard to parse programmatically, and grow without bound.
Leave Tracker (one workbook, one tab per person, 57 tabs)	Left block logs leaves: Leave Type, When, Hours, Note, plus compensation notes ("compensated 3-5 July"). Middle block totals days by category: Casual, Sick, Vacation. Right blocks hold monthly per-day "extra" and "short" durations entered by hand as hh:mm.	No computed running balance of over/under hours exists in the file. Extra/short entries are typed manually per day. Compensation offsets live in free-text notes, so nothing reconciles automatically.
Compliance sheet (one workbook, one tab per month)	Rows are employees grouped by department. One column per calendar day holding Y, N, PARTIAL, LEAVE, blank, or free text ("4 hours 30 min"). Final column: STRIKES FOR MONTH = COUNTIF(N) + COUNTIF(PARTIAL).	Every daily mark is entered by hand after cross-referencing the other two artifacts. The strike count is the only automation in the entire workflow.

The admin cross-references all three daily. The app replaces the cross-referencing: the employee logs time once, and leave records plus compliance status derive from that single entry automatically.

2. What this replaces and supersedes

- Replaces all three spreadsheets as the system of record for time, leave, and compliance.
- Supersedes the Timesheet Compliance Checker (Requirements v1). That tool read and parsed the spreadsheet files from SharePoint. With the app as the system of record there are no files to parse, so the checker's format-validation and AI-adjudication logic is unnecessary. Its status model (Complete / Short / Missing / Exception) carries forward into Section 6.
- Runs until the third-party HR software pilot concludes. Design for export, not for permanence.

3. Users and roles

Two roles for the POC as originally scoped — **superseded in production by a three-tier model**, see the status update above and README's "Screens & roles" for the current, authoritative version. Kept here for historical context.

Role	Who	Can do
Employee	~45 active staff (originally all India-based offshore; the team has since grown to include US-based staff too)	Log tasks for their own day, submit their day, view their own hours, variance balance, leave history, and strike count. Nothing else.
Admin	HR/admin (currently Norine's function), Steve, Mary	Everything: roster, dropdown lists, leave entry and approval, status overrides, strike review, thresholds, exports.

What actually shipped: Employee (`is_admin=False`) is unchanged. Admin split into two tiers — a **department-scoped Admin** (`is_admin=True, is_super_admin=False`) who gets the Dashboard, Leave/Overtime Requests, Reports, Suggestions, and Assignments filtered to their own department only, and **Super Admin** (`is_super_admin=True`) who gets everything the original single Admin role described, unscoped. Existing admins were automatically promoted to Super Admin the moment this shipped, so nobody who could already see everything lost access. A separate, independent **Developer** flag (unrelated to the admin tiers — a plain Employee can hold it) gates only the ability to change a Ticketing System ticket's status.

Team leads exist in the data (department groupings with TEAM LEAD designations) and are no longer just a fast-follow idea: an employee's **Reports to** field (Roster, restricted to people who are Admins) is who their Overtime pre-approval requests route to, and department-scoped Admin doubles as a practical team-lead view for everything else (dashboard, leave, reports) scoped to that person's own department. A dedicated read-only team-lead-only screen, as originally imagined, was never built separately — the department-scoped Admin tier covers the same need.

4. Module A: Daily time log (replaces Task Summary)

Entry

The employee's main screen is today's log. Each row:

Field	Rules
Date	Defaults to today. Employees cannot log more than 1 working day in the past (configurable, see Section 10).

Field	Rules
Project/Employer	Dropdown, admin-managed list. The current files already use dropdowns fed by a hidden List sheet (client names such as “AB2 Consulting, Inc.”, “9Logic Technologies INC”). Free text not allowed.
Task	Dropdown, admin-managed list (current values include “Check emails”, “Internal Meetings”, “PWD initiation”, “Drafting letters”, “Send email”). Free text not allowed.
Details	Free text, required, minimum 5 characters.
Start Time	Time picker.
End Time	Time picker. Must be after Start Time.
Duration	Computed from Start and End. Never typed. This kills the current cross-check problem where “Time taken” and the Start-to-End math disagree.

Validation on entry (the strict guidelines)

- No overlapping rows within a day.
- Gaps between rows over 15 minutes get a visual flag, not a block. Breaks are legitimate, so logged break time inside the gap is netted out first — only the leftover, still-unexplained minutes get flagged (added 2026-08-11, after the flag fired on a gap almost entirely covered by a logged break).
- A row cannot be logged over a time span the employee already logged as a break — blocked outright, with a message pointing at the break window, so work time and break time can’t overlap in the log (added 2026-08-11).
- No row may span past midnight; split across days.
- Maximum single-row duration 4 hours (configurable). Longer rows force the employee to break work down, which is the point of minute-level logging.
- **A rejected submission no longer resets the form** (added 2026-08-14): whatever Project/Task/Details/Start/End was typed re-shows exactly as submitted, and if the failure was specifically a time conflict, Start is auto-nudged to the earliest minute that actually clears it, instead of the employee guessing a new time by trial and error.
- **Details can be edited in place after saving**, but only for today’s own not-yet-locked rows (added 2026-08-10) — a past day, even one still inside the back-dating window, stays closed to quiet edits the same way it’s closed to deletes once submitted, so history stays trustworthy. Changing the time/project/task still means delete-and-re-add, since that would re-open overlap/cap validation.
- **Auto time capture** (added 2026-08-01): a Start/Stop Timer widget fills a row’s Start/End from the system clock instead of typed times — one active timer per employee; starting a new one auto-stops and logs whatever was already running. Runs through the exact same validation as a typed row.

Day submission

- The employee clicks Submit Day when done. Daily total is computed, never typed.
- Submission locks the day. Edits after submission require an admin unlock, and every unlock is logged (who, when, what changed).
- Unsubmitted days roll into compliance as missing (Section 6).

Breaks and Punch In/Out (added 2026-07-30 through 2026-08-14, not in the original scope)

Two related but independent live-timer features sit alongside the task log:

- **Break tracking** — Start Break / End Break (Personal, or Lunch/Dinner once per day) with a live running timer. A completed break is netted out of gap-flagging and blocked from overlapping logged task time (see validation rules above), and auto-adds a read-only “General / Break” row to the task log with an optional note — so a break is visible in the log without an employee re-typing it under a real client Project (which happened in practice and read wrong on reports). This row is display-only: the day’s logged total, target, and everything downstream (compliance, compensation, overtime, strikes) still key off real task rows exactly as before.
- **Punch In/Out** — a separate, personal countdown-to-target widget, not a task log entry. It pauses the instant a break starts and resumes correctly once it ends. Once the target is reached it flips to counting overtime. This is display-only and doesn’t gate or log work by itself, but two things key off it: an **automatic Punch-Clock compensation balance** (My Month) that’s independent of the manual compensation-link feature in Module B, and Reports → Attendance’s Overtime figure. Punch Out itself is blocked until the day’s task log has been Submit Day’d, so a punched duration is never left dangling with nothing behind it in the log.

5. Module B: Leave and hours ledger (replaces Leave Tracker)

Leave records

Admin-entered as originally scoped (see open question 5 on employee self-service requests) — **superseded**: employees can now submit their own leave requests, which an admin approves or rejects; admin can still enter already-approved leave directly, unchanged. Either path recomputes affected days immediately.

Field	Rules
Employee	From roster.
Date(s)	Single day or inclusive range.
Type	Casual, Sick, Vacation, plus Other with a note (current data contains types like “Traveling”, “Holiday”, and reasons like “power outage” that do not fit the three categories).
Hours	Full day (defaults to the person’s daily target) or partial hours.
Note	Free text.

Category totals per person (the Days Taken block) are computed, not typed. **Added 2026-08-10:**

a live leave-balance figure (remaining vs. annual entitlement per category, e.g. “8 / 10 Casual”) is shown to both the employee (Leave page) and admin (Leave Requests page), computed fresh from approved leave taken so far in the current calendar year — nothing is decremented or stored, so it resets automatically each January with no year-end job. It doesn’t block requesting or recording leave past zero remaining; it’s a visibility figure, matching the “no quotas enforced, totals displayed” default in open question 6.

Hours variance (the extra/short tracking)

Computed daily per person, no manual entry:

variance = submitted hours for the day - daily target for the day

- Daily target comes from the roster (default 8.0 hours full time; the sheets show part-time staff such as “4 Hours 19 min” days, so target is per person).
- Approved leave reduces that day’s target by the leave hours. A full-day leave means target 0 and no variance.
- The ledger shows each day’s variance and a running net balance. This is the running total the current sheet implies but never actually computes.

Compensation (make-up hours)

The current tracker handles this in free text (“compensated 3-5 July”, “compensate”). The app makes it a first-class link: an admin marks a shortfall as compensated by pointing it at one or more surplus days. Both sides display the link. The running balance already nets out mathematically; the link exists so a human can see which short day was made up when, which is what the disciplinary conversation needs.

6. Module C: Compliance and strikes (replaces the compliance sheet)

Daily status, auto-derived

Computed nightly and on demand for every active person for every working day. Never typed.

Status	Condition	Maps to current sheet
Complete	Day submitted, hours >= target minus tolerance	Y
Partial	Day submitted, hours below target minus tolerance	PARTIAL
Missing	Working day, no submission, no approved leave	N
Leave	Approved leave covers the day	LEAVE
Holiday / Weekend	Non-working day	blank

Default tolerance 1.0 hour (carried from the Compliance Checker v1 requirements, open question 2). Target and actual always display as numbers so the admin can judge borderline cases rather than trusting the threshold.

Strikes

Verified rule from the current sheet's formula: strikes per month = count of Missing days + count of Partial days, each worth one strike. The July 2026 sheet shows real counts from 0 to 10.

- Strike counts compute automatically from daily statuses. The admin's manual daily marking disappears entirely.
- Admin can override any day's status with a mandatory reason. Overrides are logged and visibly flagged, because an override changes someone's strike count and potentially their pay.
- Compensated shortfalls: whether a compensated Partial day still counts as a strike is open question 3. Default for the POC: a fully compensated day is recomputed as Complete and the original status is retained in the audit log.

Violations

The current data counts strikes but defines no threshold for action. The user rule is: X strikes = violation, repeated violations = pay reduction. Neither X nor the pay consequence appears anywhere in the files.

- POC ships with a configurable monthly strike threshold (placeholder default: 5) that flags the person on the admin dashboard as In Violation.
- The app flags. It does not calculate or apply pay changes. Pay action stays a human decision outside the tool (open question 4).

7. Screens

The nine screens originally scoped, as built. **This list has grown substantially since v1** — see Section 13 for every screen added afterward (Punch In/Out and Break are folded into Today itself, not separate screens; Overtime, Holidays, Support/Tickets, Profile, Reports ×3, Suggestions, Assignments, Bulk uploads, Audit, and Health check are all net-new). Section 13 and the README's "Screens & roles" table together are the authoritative current list; this section is kept as the historical baseline.

Employee

1. Today: task log entry, running total for the day, Submit Day.
2. My Month: calendar of own statuses, hours vs target per day, running variance balance, leave taken by category, current strike count. Employees seeing their own strike count in real time is deliberate; today they find out after the fact.

Admin

1. Compliance dashboard: the current monthly sheet, rebuilt live. Rows = employees grouped by department, columns = days, cells = auto-derived status, strike total per row. Filters: department, exceptions only, month. This view existing at all is the core payoff.
2. Person detail: one employee's full log, ledger, leave history, overrides.

3. Roster: add/deactivate people, set department, designation, daily target, work schedule, start date. Deactivated people keep history (the current tracker's GONE section) but drop out of compliance runs.
4. Lists: manage Project/Employer and Task dropdown values. Deactivating a value hides it from new entries without breaking old rows.
5. Leave entry and compensation linking.
6. Config: tolerance, strike threshold, max row duration, back-dating window.
7. Export: any view to CSV/XLSX. This is the bridge to the third-party software and the safety net if the POC dies.

8. Data model

Entities, minimum fields, as originally scoped (the build team owns the schema details — kept here as the historical baseline; the live schema in `app/models.py` is authoritative and has grown well beyond this list to cover every module in Section 13, e.g. `BreakEntry`, `PunchSession`, `OvertimeApproval`, `Ticket/TicketComment`, `Holiday`, `EmployeePersonalDetails/EmployeeBankDetails`, `ActiveTaskTimer`, `ProjectAssignment/TaskAssignment`):

Entity	Key fields
Employee	id, name, department, designation, daily_target_hours, schedule (work days, standard hours, timezone), start_date, active
TaskEntry	id, employee_id, date, project_id, task_type_id, details, start_time, end_time, duration (computed), created_at
DaySubmission	employee_id, date, total_hours (computed), submitted_at, locked, unlock_log
LeaveRecord	id, employee_id, date_range, type, hours, note, entered_by
DayStatus	employee_id, date, status, actual_hours, target_hours, variance, override (reason, by, at, original_status)
CompensationLink	shortfall_day, surplus_day(s), linked_by, note
Project, TaskType	id, name, active
Config	tolerance, strike_threshold, max_row_hours, backdate_days
AuditLog	actor, action, entity, before, after, timestamp

Statuses and strikes are computed views over `TaskEntry`, `DaySubmission`, and `LeaveRecord`, materialized into `DayStatus` for speed and override support. Nothing about compliance is ever hand-entered except overrides. All durations/targets/variances are stored as integer **minutes**, not the hour-decimal shown in this table's original field names — that convention was set during the build and has held throughout.

9. Build guidance for the POC

Suggestions, not mandates, as originally written — **what actually shipped is noted inline**, and the status update at the top of this document has the fuller picture.

- Single web app, one codebase, role-gated views. Python (FastAPI or Django) or Node, server-rendered or light React. Nothing fancy. — **Shipped as:** FastAPI + SQLAlchemy + Jinja2, server-rendered, no JS framework/build step.
- Postgres. Not SQLite, because ~45 concurrent users submit at end of day, and not files, because files are the disease being cured. — **Shipped as scoped.** SQLite remains the zero-setup local-dev default; DATABASE_URL switches to Postgres with no code changes. Production runs on Cloud SQL for PostgreSQL.
- Auth through the existing Microsoft tenant (Entra ID) so there are no new passwords and access is revocable centrally. Same model as the compliance checker plan. — **Superseded:** self-signup password auth (AUTH_MODE=password) shipped instead — an admin creates the roster row, the employee claims it via /signup against their own email, rate-limited logout after repeated failures. Entra ID remains a designed-for, not-yet-used swap point in app/auth.py (AUTH_MODE=entra) if the org standardizes on it later.
- Host on Azure App Service inside the current tenant, matching the email system and checker architecture already approved. — **Superseded:** production runs on Google Cloud Platform (Cloud Run + Cloud SQL for PostgreSQL), not Azure — the team stood up its own GCP project rather than waiting on the org's Azure tenant. An equivalent Azure App Service deployment path is kept in the repo (deploy_azure.sh) as a documented migration option, not the live one.
- Seed script that imports the three existing files: roster and departments from the compliance sheet, leave history from the tracker, and task history from the Task Summary files where parseable. Historical strike counts from March through July 2026 should reproduce the sheet's numbers; that reproduction is the acceptance test for the status engine. — **Shipped as scoped**, and still the acceptance gate: legacy/import_legacy.py + legacy/verify_strikes.py, 168/168 person-months, must stay green on any engine/importer change.

10. Open questions

Defaults let the build start. Every one of these is a config value or a one-line change.

#	Question	POC default	Current answer, if it changed
1	How many strikes in a month = violation, and what escalation follows? Not defined anywhere in the current data.	Threshold 5, dashboard flag only	Unchanged — still config-only, still a dashboard flag, no automatic escalation.
2	Tolerance below target before a day counts as Partial?	1.0 hour	Unchanged (tolerance_minutes config, 60).

#	Question	POC default	Current answer, if it changed
3	Does a compensated shortfall erase the strike?	Yes, with audit trail	Unchanged (<code>comp_erases_strike</code> config, on by default).
4	Should the app model pay reductions at all?	No, human decision	Unchanged.
5	Do employees request leave in-app with admin approval, or does admin enter everything?	Admin enters (matches today)	Changed: both — employees can self-submit a request for admin approval/rejection, admin can still enter already-approved leave directly. A live per-type leave balance (annual entitlement minus approved leave taken this calendar year) shows to both employee and admin.
6	Leave quotas per category per year? None appear in the tracker.	No quotas enforced, totals displayed	Unchanged — the new leave-balance figure (see #5) is visibility only, doesn't block requesting/recording past zero remaining.
7	How far back can an employee log or edit an unsubmitted day?	1 working day	Unchanged in principle (<code>backdate_working_days</code> config); "edit" was clarified in practice to mean <i>adding a new row</i> for a past date, not editing an already-logged one — an already-logged row is only ever editable (Details text only) on the day it was logged, before it's locked.

#	Question	POC default	Current answer, if it changed
8	Are weekends universally non-working? Compliance sheet leaves them blank, but Prem's history shows weekend and pre-shift logging.	Mon-Fri working, per-person schedule override available	Unchanged.
9	Company holiday calendar source and regional scoping?	Admin-maintained table, single region	Tried and reverted: briefly split by employee country (US/India, <code>location</code> field on Employee/Holiday) starting 2026-08-12, after the team grew to include both US and India staff. Reverted 2026-08-14 back to the original one-shared-calendar-for-everyone design — the per-country split turned out not to be what the team actually wanted once it shipped. The <code>location</code> field itself stays in the schema (Employee/Holiday) for safety — dropping a column isn't a safe change once real production data exists — but nothing reads it for holiday scoping anymore; <code>holidays_set()</code> always returns every holiday, unscoped.

11. Out of scope for the POC

- Anomaly detection (pre-shift work, meta-work ratios, unauthorized projects, the Prem-style analysis). Phase 2, same as the checker plan. The structured data this app produces makes that analysis trivial later, which is a reason to build this even with the third-party pilot running. **Still out of scope.**
- Payroll integration or pay calculations. **Still out of scope.**
- Mobile apps. Responsive web is enough. **Still out of scope** — no native app; the web app is responsive.
- Notifications (end-of-day reminders, strike alerts). Fast follow. **Still out of scope.**
- Team-lead role and views. **Delivered, in a different shape than imagined:** rather than a dedicated read-only team-lead screen, a department-scoped Admin tier covers the same need (dashboard/leave/reports filtered to one department), and a per-person “Reports to” field independently routes Overtime pre-approval requests to that person’s actual lead. See Section 3 and Section 13.
- Migration of full task history from every legacy file. Roster, leave, and strike history migrate; task-level history migrates best-effort. **Still true**, unchanged since v1.

Genuinely new scope, not imagined in v1 at all, that’s since shipped: break tracking, Punch In/Out with automatic compensation, overtime pre-approval, a full ticketing system, employee self-service leave requests, personal/bank details self-service, bulk admin tooling beyond the roster, and three dedicated reporting pages. See Section 13.

12. Success criteria

1. An employee logs and submits a full day in under the time the spreadsheet takes, with zero manual arithmetic. **Met.**
2. The admin’s daily cross-referencing task is eliminated: the compliance dashboard shows every person’s status for yesterday with no data entry. **Met.**
3. Recomputed March-July 2026 strike counts match the existing sheet’s STRIKES FOR MONTH column for migrated data. **Met — 168/168, and still gated on every engine/importer change via `legacy.verify_strikes`.**
4. Any admin view exports cleanly to XLSX for the third-party pilot handoff. **Met**, and expanded — Dashboard, Person Detail, all three Reports pages, and a raw org-wide CSV dump all export.

Additional criteria met beyond the original four, reflecting what the app actually had to prove out once real people used it daily:

5. The app runs on real, persistent production infrastructure (GCP Cloud Run + Cloud SQL for PostgreSQL) with real authentication (self-signup password auth, rate-limited), not just a local demo. **Met.**
6. Schema changes are additive-only once real production data exists — no more casual `rm tms.db + re-import` once the team is actually using the system daily. **Met and enforced as a hard rule** — see `app/db.py`’s additive-migration guard and `CLAUDE.md`.
7. Feature work can ship incrementally without breaking what’s already live — demonstrated by the Holiday Management per-country split being tried, shipped, and cleanly reverted within days via query/business-logic changes alone, no destructive schema change required either way. **Met.**

13. Modules added since v1 (not in the original nine-screen scope)

Everything below shipped after the original build, driven by direct manager/employee feedback rather than a new requirements round. Grouped by module; each links back to where it's implemented for anyone extending it further. See the README's "Screens & roles" section for the exhaustive, always-current version — this section is the PRD-level summary.

Break tracking & Punch In/Out

See Section 4's "Breaks and Punch In/Out" subsection above.

Overtime pre-approval

An employee requests pre-approval to work overtime over a future (or, for an admin acting directly, past) date range with a note. Routed to whoever their **Reports to** (Team Lead) is; if nobody's assigned, any Super Admin can act on it. Doesn't block anything — an employee can still log time and use Punch In/Out with or without approval; it's a payroll-visibility label. Reports → Attendance shows both the raw overtime figure and, separately, an "Approved" figure counting only overtime inside an approved date range, so whoever runs payroll can see what's actually payable. Doesn't touch DayStatus, strikes, or compensation links.

Ticketing System

Anyone who can log in can raise a bug/enhancement/new-feature ticket (with priority and an optional image/video attachment), view every ticket org-wide, open its detail page, and comment. A separate, independent **Developer** flag (a plain Employee can hold it; an Admin need not) gates only the ability to change a ticket's status (Open → In Progress → Resolved/Closed). Built and fully tested; currently held behind a feature flag (TICKETING_ENABLED) pending its own dedicated rollout, deliberately sequenced after the Time by Project/Task report shipped on its own first.

Reports

Three cascading-filter report pages (Department → Employee → date range), each exporting to XLSX: **Attendance** (per-person attendance % or day-by-day detail, plus raw and approved overtime), **Strikes** (per-person strike counts or day-by-day detail), and **Time by Project/Task** (logged time per employee broken into monthly trend columns, independently filterable by Project and/or Task so an admin can answer "how much time went into this client" or "how much time did the team spend on this kind of work across every client"). Department-scoped admins see only their own department in all three.

Self-service profile

Beyond the roster-managed fields from Section 8: a profile photo, a **Personal Details** card (DOB, contact info, family, nationality, hobbies/skills/languages), and an **Employment Details** card (bank account, PAN/Aadhaar/UAN/ESI) with every sensitive number masked to last-4 everywhere it's shown, including to admins, the instant it's saved. Both optional and employee-editable; admin gets a read-only view once filled in.

Company Holiday Management

An admin-maintained calendar (add inline or bulk-upload via Excel — just Name and Date), one shared list every employee's working-day/compliance calculation uses. Resolves open question 9 — see that entry above for the per-country detour and reversal.

Bulk admin tooling beyond the roster

Excel upload for: roster onboarding/updating/bulk-offboarding (the original scope), plus **leave-quota allocation** (bulk-set annual Casual/Sick/Vacation entitlement per employee), **Project/Task dropdown values** (single-column, add-only, so a long legacy list can be seeded quickly), and **Holiday calendar entries**. Each follows the same overwrite-on-reupload pattern: blank cells leave existing data untouched, a template and a current-data export are both one click away.

Support questions

A simple ask-an-admin-a-question channel, distinct from the Ticketing System — for “how do I...” and account issues rather than bugs/feature requests.